



Projet informatique — 2^e année

LaborScope

Dossier d'analyse

Étudiant-e-s

Jade COLIN
Sidy Mohamed Salim DIALLO
Alexandre FRATTER-BARDY
Adam Ouangko HAÏWÈ
Vicram RAVEENDIRAN

Responsable

Ludovic DENEUVILLE

Encadrant

Anas KNEFATI

Table des matières

1	Introduction	2
2	Cahier des charges	2
3	Questionnements et choix	3
4	Description des fonctionnalités	4
5	Organisation du groupe	5
6	Organisation du travail	5
7	Modèle de conception	5
7.1	Diagramme de cas d'utilisation	6
7.2	Diagramme d'activité	8
7.3	Diagramme de classes	9
7.4	Diagramme de séquences	10
7.5	Diagramme de séquences : authentification	10
7.6	Diagramme de séquences : données analytiques	11
7.7	Diagramme de packages	12
8	Liste des principaux composants	12
8.1	Les DAO	12
8.2	La couche de services	13
8.3	L'API	13
8.4	Le frontend	13

1 Introduction

Le marché du travail occupe une place centrale dans nos sociétés. Que ce soit pour suivre l'évolution de l'emploi, comprendre les différences entre les pays ou encore analyser les inégalités entre les catégories de population, les statistiques liées au travail constituent une source d'information essentielle. En effet, l'emploi peut être influencé par de nombreux facteurs tels que l'âge, le sexe, la profession ou encore la situation économique d'un pays. L'analyse de ces données permet ainsi de mieux comprendre les évolutions du marché du travail et les différentes tendances qui peuvent apparaître au fil du temps.

Cependant, la quantité de données disponibles peut rendre leur utilisation assez complexe. Les statistiques sont régulièrement mises à jour et peuvent être classées selon de nombreux critères. Il peut alors être difficile de retrouver une information précise, de comparer plusieurs pays ou encore de suivre l'évolution d'un indicateur sur une période donnée. Disposer d'un outil capable de regrouper ces données, de les traiter et de les présenter de manière simple permettrait donc de faciliter leur analyse.

C'est dans cette idée que s'inscrit notre projet LaborScope. Son objectif est de développer une API permettant d'exploiter et d'analyser des statistiques mondiales sur le marché du travail, en utilisant notamment les données ouvertes de l'Organisation Internationale du Travail disponibles via l'API ILOSTAT. Ces données permettent par exemple d'étudier l'emploi en fonction du sexe et de la profession, mais aussi la population en âge de travailler selon le sexe et l'âge, pour de nombreux pays et régions du monde.

Le projet intégrera également une gestion des utilisateurs avec différents niveaux d'accès. Des fonctionnalités complémentaires pourront être ajoutées, comme une cartographie interactive ou la génération automatique de rapports. L'objectif de LaborScope est de proposer une solution simple et évolutive pour faciliter l'analyse des données du marché du travail. Dans ce rapport, nous présenterons le cahier des charges, la conception et l'architecture de l'application, les choix techniques ainsi que la réalisation des principales fonctionnalités.

2 Cahier des charges

Notre application doit comporter plusieurs fonctionnalités essentielles. L'utilisateur devra ainsi pouvoir accéder :

- à la récupération et à la mise à jour des données provenant de l'API ILOSTAT ;
- à la consultation des données concernant différents pays, sexes, tranches d'âge et professions ;
- à l'analyse de l'évolution d'un indicateur sur une période donnée ;
- à la comparaison d'un indicateur entre plusieurs pays ;
- à la comparaison de plusieurs indicateurs sur une même période ;
- à des graphiques permettant de visualiser clairement les résultats ;
- à un système d'authentification avec différents niveaux d'accès (utilisateurs et administrateurs)

Les administrateurs disposent également de fonctionnalités supplémentaires leur permettant de gérer les données stockées et de suivre les activités de l'application. Par ailleurs, plusieurs fonctionnalités optionnelles sont envisagées afin d'enrichir l'expérience utilisateur, telles que :

- l'affichage des indicateurs sur une carte interactive du monde ;
- la génération automatique de rapports d'analyse au format PDF ;

Ces fonctionnalités supplémentaires pourront être développées en fonction de l'avancement du projet et seront détaillées davantage dans le dossier final si elles sont retenues.

3 Questionnements et choix

Lors de la phase de conception de LaborScope, plusieurs réflexions ont été menées afin de définir les orientations techniques et fonctionnelles du projet.

Le premier choix décisif a concerné l'architecture globale de l'outil. Bien que le cahier des charges laisse la possibilité de se limiter au développement de l'API, nous avons pris le parti de développer la solution la plus complète : une API couplée à une interface graphique (front-end). Cette approche nous paraît indispensable pour répondre à l'objectif principal du projet, c'est-à-dire rendre des volumes importants de statistiques accessibles, exploitables et visuellement clair pour l'utilisateur final.

Une autre réflexion majeure a porté sur le système d'authentification et la répartition des droits d'accès. Plutôt que d'imposer la création d'un compte de manière stricte dès l'ouverture de l'application, nous avons opté pour une approche progressive. L'objectif est de démontrer l'utilité de notre outil dès la première visite.

Ainsi, nous avons décidé de laisser la fonctionnalité d'analyse temporelle de l'emploi (F3) en accès libre. Un visiteur non connecté pourra donc interroger la base de données, générer un graphique et observer l'évolution d'un indicateur sans restriction.

En revanche, l'accès aux fonctionnalités d'analyse plus avancées (comparaison géographique, croisement multi-indicateurs, cartographie ou génération de rapports) nécessitera obligatoirement une authentification. Ce choix stratégique permet d'offrir un aperçu concret des capacités de l'application pour susciter l'intérêt de l'utilisateur, tout en l'incitant à s'inscrire pour bénéficier d'une expérience complète.

Nous avons fait le choix d'orienter l'application vers une logique marketing et commerciale afin d'attirer et de fidéliser de potentiels clients comme dans une vraie application. Cette dynamique permet d'enrichir notre base de données clients et s'intègre parfaitement avec le système de gestion des rôles (Utilisateurs et Administrateurs) mis en place pour administrer l'application.

4 Description des fonctionnalités

Le projet comporte différentes fonctionnalités qui se répartissent entre fonctionnalités obligatoires et fonctionnelles. Il y a 6 fonctionnalités obligatoires ainsi que 2 fonctionnalités optionnelles.

La première fonctionnalité de cette application consiste en la récupération et le stockage automatisés des données. Ainsi, l'application interroge automatiquement et de façon périodique l'API ILOSTAT et récupère les indicateurs souhaités sur plusieurs pays. Ensuite vient le traitement des données, les fichiers bruts sont triés pour ne conserver que les champs jugés essentiels au bon fonctionnement de l'application. Ces données nettoyées sont ensuite sauvegardées dans la base de données locale et deviennent enfin prêtes à être analysées.

Une autre fonctionnalité doit permettre l'analyse temporelle des données extraites et stockées. L'utilisateur pourra sélectionner un indicateur, définir une fenêtre de temps et afficher les résultats sous forme graphique. Concrètement, après la sélection des paramètres, l'API calcule cette évolution temporelle et l'interface affiche permet d'observer les dynamiques du marché du travail.

Outre la comparaison temporelle d'un indicateur, cette fonctionnalité propose à l'utilisateur, pour la période la plus récente possible présente dans la base de données, de comparer un indicateur entre plusieurs pays. L'utilisateur peut choisir une liste de pays ainsi qu'un indicateur et l'application met en évidence les pays présentant les valeurs les plus hautes et les plus basses concernant cet indicateur.

La fonctionnalité de comparaison multi-indicateurs doit permettre à l'utilisateur de croiser plusieurs jeux de données. L'utilisateur pourra superposer plusieurs indicateurs sur un même graphique ainsi qu'une échelle de temps. Cela permet de dégager visuellement des liens entre les indicateurs sur une période donnée et ainsi de comprendre quels mécanismes régissent le monde du travail.

La dernière fonctionnalité obligatoire consiste en la gestion des utilisateurs et de la sécurité. Ainsi, l'accès à l'application sera sécurisé par une authentification vérifiant les mots de passe et le système devra gérer deux niveaux de droits d'accès. Les utilisateurs standard consultent les analyses et les visualisations tandis que les administrateurs possèdent en plus de la consultation des droits de gestion sur les données. De plus, chaque modification sur les bases de données entraîne une actualisation du journal de suivi des activités.

La première fonctionnalité optionnelle est une cartographie interactive où l'utilisateur choisit un indicateur et les données sont ensuite projetées sur une carte du monde. Chaque pays est colorié selon la valeur de l'indicateur choisi. Cela permet donc de visualiser des tendances mondiales sur un indicateur précis.

L'application intègre également une fonctionnalité permettant de générer un rapport d'analyse au format PDF. L'utilisateur sélectionne des indicateurs, une période de temps ainsi que des pays et l'outil en fera un résumé statistique contenant des graphiques et une conclusion synthétique générée automatiquement à partir des données observées.

5 Organisation du groupe

ICI mettre diagramme de Gantt

6 Organisation du travail

Concernant l'organisation du travail, notre équipe s'appuie sur plusieurs outils collaboratifs afin d'assurer une communication fluide et une répartition efficace des tâches. Pour nos échanges quotidiens, nous disposons d'un groupe de discussion sur WhatsApp, ce qui nous permet de débattre de nos idées et de faire des points réguliers sur l'avancement du projet. En parallèle, nous utilisons Google Docs pour la rédaction et le partage de nos documents communs garantissant ainsi à chaque membre un accès simultané pour la lecture et la modification des fichiers.

La gestion de projet et le suivi des tâches sont quant à eux centralisés sur la plateforme Notion. Cet espace de travail nous permet de répertorier l'intégralité des actions à mener. Chaque tâche y est qualifiée selon plusieurs critères : son statut (à faire, en cours ou terminé), la phase du projet à laquelle elle se rattache, son niveau d'importance, ainsi que le membre du groupe qui en est responsable. Cette méthode structure notre organisation, offre une vision claire de la progression globale et permet à chacun de savoir précisément ce qu'il doit accomplir.

Enfin, concernant la partie développement, nous codons notre application sur VSCode et nous nous appuyons sur GitHub pour centraliser notre travail. Ce répertoire commun est très utile : il sécurise notre collaboration, nous permet de coder en parallèle, de conserver tout l'historique de nos modifications et de résoudre facilement les éventuels conflits de versions.

7 Modèle de conception

Cette phase de conception est très importante dans la mesure où elle pose les jalons du projet. En effet, cela nous permet de savoir ce que nous allons faire ainsi que les méthodes que nous allons utiliser pour y parvenir. Cette partie utilise des outils de modélisation UML pour définir tous les aspects de l'application. Par ailleurs, cette phase préparatoire de modélisation permet une meilleure répartition des tâches entre les membres du groupe concernant les travaux à mener. Ainsi, chacun sait où en est rendu le projet et ce qu'il reste à faire pour le compléter. Toutefois, cette phase de conception peut être amenée à être modifiée au cours du projet car de nouveaux éléments peuvent toujours être considérés. Il est néanmoins de la plus haute importance de présenter dès le début du projet un cadre clair que nous nous efforcerons de suivre.

7.1 Diagramme de cas d'utilisation

Ce diagramme de cas d'utilisation permet d'illustrer les différentes actions que les utilisateurs de l'application peuvent réaliser. Ainsi, le but de ce diagramme est de visualiser toutes les actions possibles et de les attribuer aux bons acteurs.

Il existe dans ce diagramme deux acteurs différents.

Le profil "Utilisateur" est le rôle standard. Il est strictement limité à l'authentification ainsi qu'à la consultation des analyses et des visualisations telles que l'analyse temporelle, la comparaison géographique ou encore la comparaison multi-indicateurs. Il bénéficie également d'outils tels que la cartographie et la génération de rapports. Les limites du profil "Utilisateur" s'opposent donc aux compétences du profil "Administrateur" qui a un rôle dédié aux tâches de gestion de l'application. Il est le seul à pouvoir modifier ou supprimer certaines données et à gérer l'accès des profils "Utilisateur". De plus, la flèche d'héritage reliant l'Administrateur à l'Utilisateur indique que l'Administrateur hérite automatiquement de toutes les fonctionnalités dont l'Utilisateur a accès.

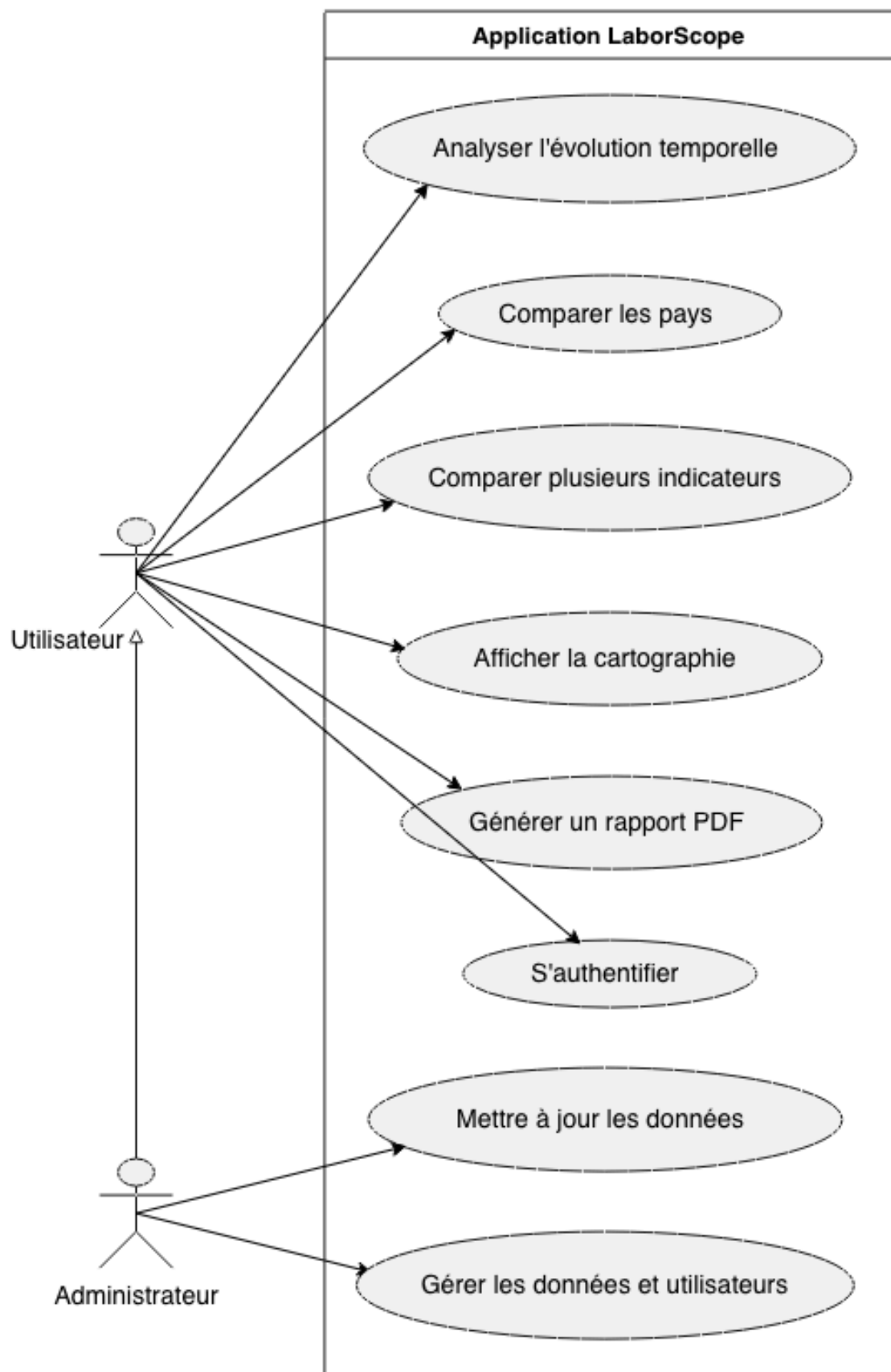


Figure 1. – Diagramme de cas d'utilisation

7.2 Diagramme d'activité

Le diagramme présenté permet de caractériser les choix offerts à l'utilisateur lorsqu'il se ouvre l'application. Dans un premier temps, il va soit se connecter, soit créer un compte s'il n'en a pas.

S'il est connecté en tant qu'utilisateur, il aura la possibilité d'accéder aux trois fonctionnalités principales :

- Calculer l'évolution d'un indicateur
- Comparer un indicateur entre pays
- Comparer simultanément l'évolution de plusieurs indicateurs

Ensuite l'utilisateur aura la possibilité en fonction des fonctionnalités choisies, de rédiger un rapport analytique ou de réaliser une carte choroplèthe.

S'il est connecté en tant qu'administrateur, il aura accès à toutes les fonctionnalités de l'utilisateur. Il pourra de plus :

- Suivre l'activité des utilisateurs
- Supprimer ou promouvoir un compte en admin
- Gérer les données

À la fin, l'utilisateur pourra se déconnecter.

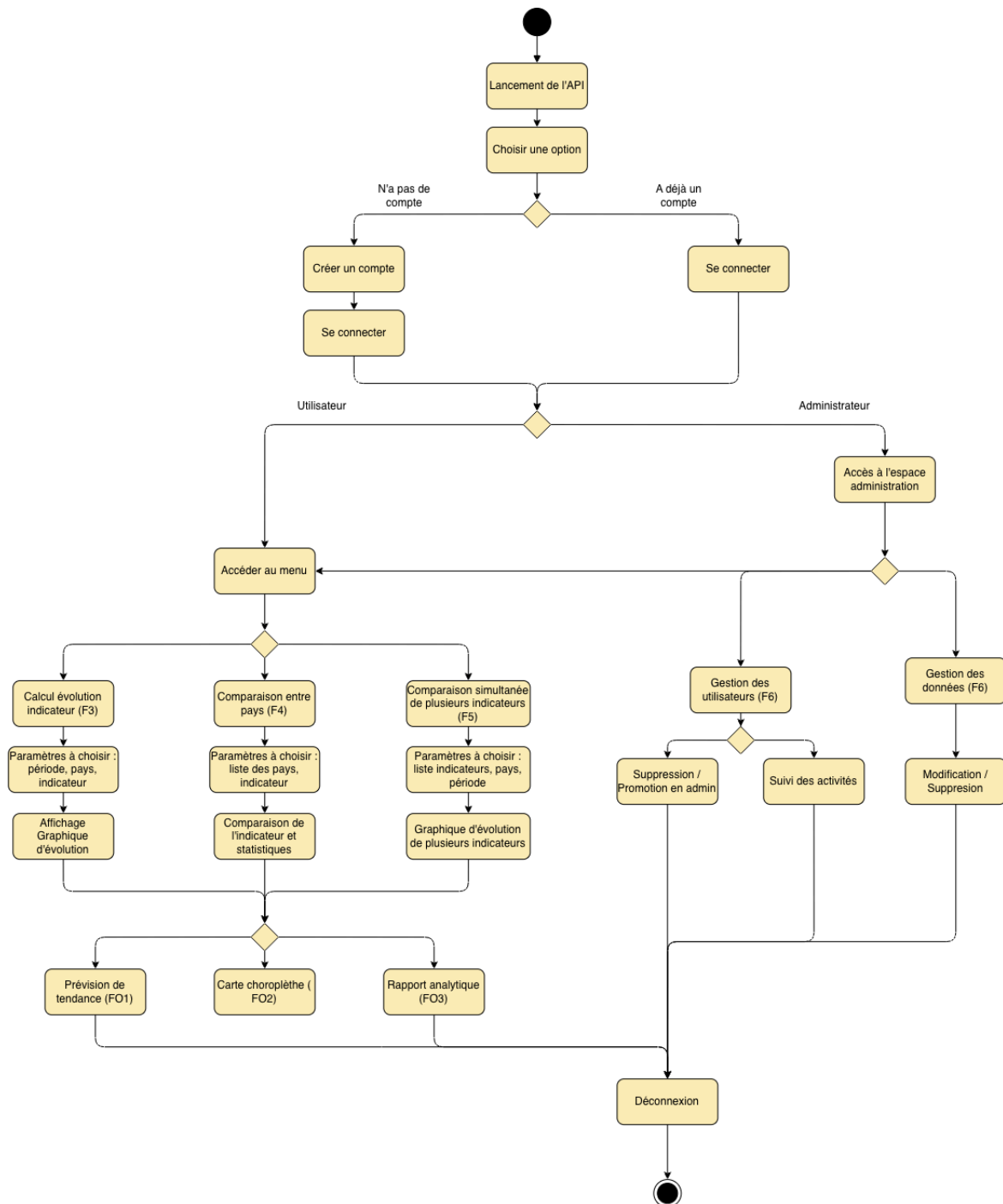


Figure 2. – Diagramme d'activité

7.3 Diagramme de classes

Cette partie présente le diagramme de classes. Notre modélisation comporte cinq classes, voici une liste de chaque classe avec une courte description pour chacune :

- **Utilisateur** : cette classe permet de gérer les connexions. Elle stocke les informations nécessaires à l'authentification et à la sécurité (identifiant, nom d'utilisateur et son mot de passe) ainsi que le rôle de la personne qui s'identifie ;

- **Journal_activite** : cette classe répond à la fonctionnalité qui exige le suivi des activités de l'Administrateur s'il modifie ou supprime des données. Il s'agit donc d'un historique où chaque action réalisée est associée à l'Administrateur correspondant ;
- **Pays** : Cette classe sert à stocker les données géographiques. Elle contient le code du pays ainsi que son nom complet ;
- **Indicateur** : Cette classe sert à savoir de quel indicateur issu de l'API ILOSTAT on parle. Elle comprend le code de l'indicateur ainsi que sa description.
- **Donnee_emploi** : Cette classe stocke la valeur chiffrée d'une statistique selon les paramètres choisis (période, sexe, tranche d'âge, pays).

7.4 Diagramme de séquences

Un diagramme de séquence permet de montrer comment les objets et les acteurs du projet interagissent entre eux selon un ordre chronologique dans le cadre d'une fonctionnalité du diagramme de cas d'utilisation.

7.5 Diagramme de séquences : authentification

On retrouve dans cette partie le diagramme de séquence concernant la fonctionnalité de l'authentification. Ce diagramme illustre l'ordre chronologique des interactions entre la personne qui tente de se connecter et les trois couches du système que sont l'interface, l'API ainsi que la base de données. L'objectif de ce diagramme est de visualiser comment l'application sépare, en fonction du profil renseigné, les droits de chaque profil (Utilisateur ou Administrateur).

Le processus d'authentification est initié par une personne qui saisit son nom et son mot de passe directement sur l'interface. Afin de maintenir la sécurité de l'architecture, l'Interface ne communique jamais directement avec les données, elle formule une demande d'authentification qu'elle transmet au Serveur. C'est l'API qui porte le rôle de la vérification. En effet, le Serveur interroge la Base de données locale pour vérifier si les informations transmises correspondent à un compte existant. Si le compte existe, la Base de données retourne à l'API l'identité de la personne ainsi que son rôle exact, à savoir s'il est simple Utilisateur ou bien Administrateur. En fonction de la réponse de la Base de données, l'application fait face à trois scénarios distincts selon le niveau d'authentification. En effet, selon que le profil renseigné soit utilisateur ou Administrateur, le cahier des charges nous indique que l'utilisateur ne doit pas bénéficier de tous les accès. Voici la liste des trois scénarios :

- Si le rôle détecté est "Utilisateur", l'API confirme la réussite de l'authentification à l'Interface et accorde ensuite un accès restreint à cet Utilisateur (consultation des analyses et des visualisations)
- Si le rôle détecté est "Administrateur", l'API confirme la réussite de l'authentification à l'Interface et accorde ensuite tous les accès de l'application à ce profil
- Si la vérification échoue, l'API notifie le refus de connexion à l'Interface et bloque l'accès à l'application en affichant un message d'erreur.

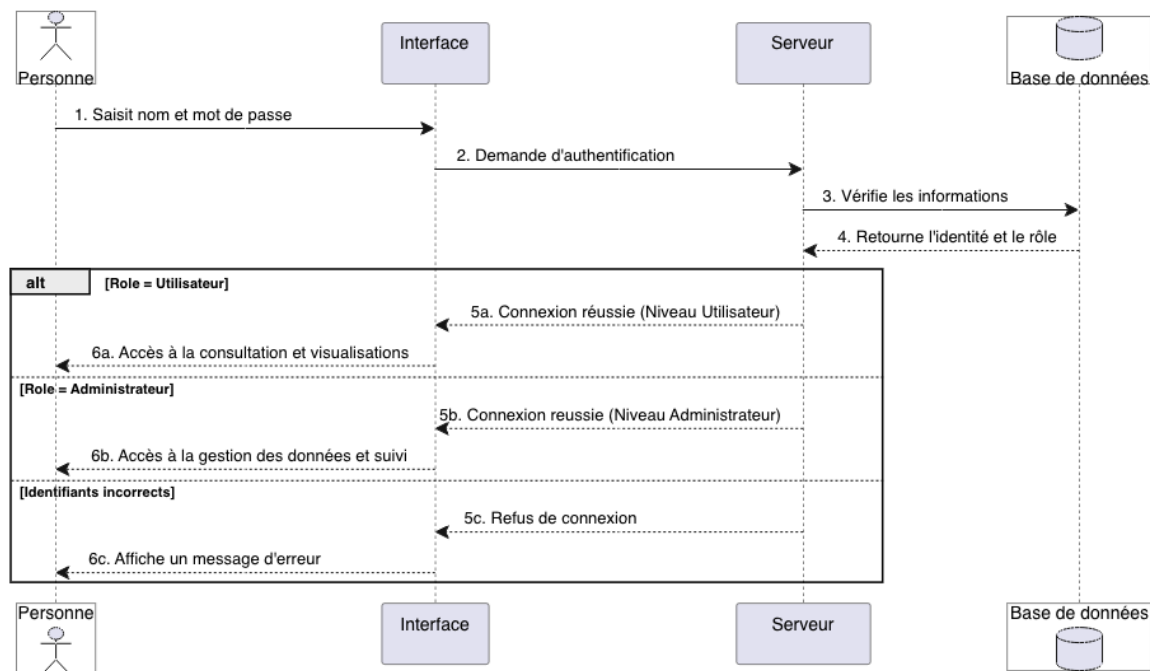


Figure 3. – Diagramme de séquences (authentification)

7.6 Diagramme de séquences : données analytiques

Cette partie présente le diagramme de séquence qui regroupe les fonctionnalités d'analyse de l'application telles que l'analyse temporelle, la comparaison géographique d'un indicateur ou encore la comparaison multi-indicateurs. Même si les calculs statistiques cachés derrière ces fonctionnalités diffèrent, le cheminement informatique reste similaire entre ces fonctionnalités.

Ainsi, pour ces fonctionnalités, le processus est déclenché par l'utilisateur qui depuis l'Interface sélectionne les paramètres d'analyse : indicateur, pays, période de temps. Une fois la demande d'analyse validée, l'Interface formule une requête qu'elle envoie à l'API. À la réception de cette requête, l'API prend le relais concernant la gestion des données et interroge la Base de données locale avec les critères de filtrage renseignés par l'utilisateur. Ensuite la Base de données répond à cette requête en retournant les données correspondantes. La dernière étape consiste en la préparation et l'affichage des résultats. Pour cela, l'API nettoie les données brutes et calcule les statistiques demandées par l'utilisateur avant de renvoyer ces données formatées à l'Interface. À la réception de ces données finales, l'Interface les utilise pour construire le graphique approprié et l'affiche à l'utilisateur.

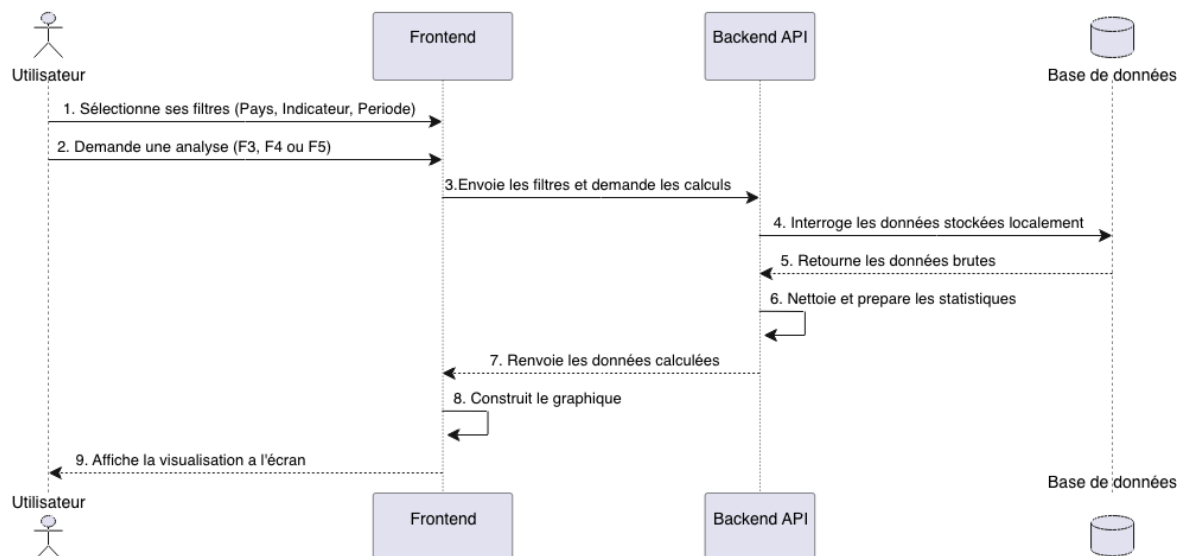


Figure 4. – Diagramme de séquences (données analytiques)

7.7 Diagramme de packages

Le diagramme de packages nous permet de voir comment les différentes couches du code interagissent entre elles. La première couche est le frontend, et permet à l'utilisateur d'interagir avec l'application. Il pourra se connecter et ensuite naviguer dans l'application. La deuxième couche est l'API, qui va transmettre les demandes, commandes de l'utilisateur vers le programme. La couche de services va réceptionner ces demandes pour ensuite réaliser des calculs. Cette couche comprend tous les objets, classes relatives aux utilisateurs et calculs à effectuer. Elle interagit avec la dernière couche qui est la couche de données. Les données sont récupérées et parsées, pour être envoyées à la DAO. Les données seront alors traitées pour ensuite être stockées localement dans une base de données PostgreSQL. Ces données seront donc exploitées afin d'effectuer les calculs dans la couche de services. Enfin, on pourra réaliser des visuels graphiques dans la couche de services, qui apparaîtront directement dans le frontend.

8 Liste des principaux composants

8.1 Les DAO

Le rôle de la couche DAO (Data Access Object) est de récupérer les données de la base de données locale. Cette couche assure le relai technique entre la base de données locale et la couche de services. Concernant ses dépendances, ce composant est totalement indépendant des autres parties de l'application. Son seul lien direct est avec la base de données locale, qu'il a justement pour mission de manipuler. À l'inverse, le serveur principal dépend entièrement de lui pour pouvoir retrouver leurs informations. Ainsi, de par son importance technique, ce composant est à développer avec une priorité absolue avec un temps de développement estimé à 40H.

8.2 La couche de services

La couche de services contiendra les classes et objets métiers. C'est cette couche qui récupérera les requêtes via l'API et qui les traitera à l'aide des fonctions implémentées. Les principales fonctionnalités seront l'affichage de l'évolution d'indicateurs, la comparaison entre pays et la comparaison de plusieurs indicateurs. Elle interagit avec la DAO pour la récupération de données utiles aux calculs. Le temps de développement est estimé à 35H.

8.3 L'API

L'API (Application Programming Interface) va nous permettre de lier le frontend et les couches métier. Elle reçoit les requêtes HTTP, les valide et communique avec les classes métier nécessaires. Elle ne sert pas à réaliser les calculs mais permet de gérer les appels du frontend, notamment sur l'authentification et l'exécution de fonctionnalités. Cette articulation sera faite à l'aide de contrôleurs. Une étape de validation est nécessaire sur l'authentification afin de valider ou refuser l'accès à un utilisateur. La couche API sera développée avec FastAPI et le temps de développement est estimé à 20H.

8.4 Le frontend

Le dernier composant que nous allons développer est le frontend. Il correspond à la couche d'interaction entre l'utilisateur et le programme. C'est là-dessus que l'utilisateur pourra faire des demandes et obtenir des résultats (visualisation graphique, calculs de statistiques...). Le package streamlit sera utilisé afin de développer cette couche. Il affichera donc l'écran de connexion et les différentes fonctionnalités que l'utilisateur pourra choisir. Cette couche interagit avec le backend qui va effectuer les calculs. Le frontend dépend donc du backend pour obtenir les données, les commandes et les résultats. Le backend quant à lui aura besoin du frontend pour enregistrer les demandes de l'utilisateur et y répondre. Le temps estimé pour développer ce composant est de 15H.